MongoDB – Collections – Basic Operations on Collections (CRUD)

What is MongoDB?

MongoDB is a **NoSQL document-oriented database** that stores data in **JSON-like documents**.

1. `show dbs`

- It **lists all existing databases** present in the MongoDB server.
- Shows **database name + size**.

```
C:\Users\sugas>mongosh
Current Mongosh Log ID: 6960e0689766b6d8f41e2620
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2
.5.10
Using MongoDB:          8.2.3
Using Mongosh:          2.5.10

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-01-09T14:00:36.827+05:30: Access control is not enabled for the database. Read and write ac
iguration is unrestricted
------

test> show dbs;
admin   40.00 KiB
config  48.00 KiB
local   40.00 KiB
test>
```

- **admin** → System database (authentication & admin tasks)
- **config** → Stores cluster/configuration info
- **college** → User-created database (appears only if it has data)

A database appears in `show dbs` **only after at least one document is inserted**.

What is a Collection?

- A **collection** is a group of **documents**
- Equivalent to a **table** in RDBMS
- **Schema-less** (documents can have different fields)

```
Database (college)
└── Collection (students)
    └── Document (student record)
```

2. Creating Database

Step 1: Switch to Database (Create Intent)

```
test> use college
switched to db college
```

This **selects** the database
Database is **not yet created**

Step 2: Check Databases
show dbs

```
college> show dbs
admin   40.00 KiB
config  36.00 KiB
local   40.00 KiB
```

`college` will **NOT appear yet**

Step 3: Create Collection + Insert Data

```
college> db.students.insertOne({
... name: "Ann",
... dept: "CSE",
... marks: 89
... })
```

Now MongoDB **creates the database physically**

Step 4: Verify Database Creation

```
college> show dbs
admin    40.00 KiB
college  40.00 KiB
config   96.00 KiB
local    40.00 KiB
```

In MongoDB, a database is created with the use <dbname> command and is physically created only after data is inserted.

3. CRUD Operations in MongoDB
CREATE Operation (Insert Documents)
   - Insert One Document

```
college> db.students.insertOne({ name: "Ren", marks: 85 })
{
  acknowledged: true,
  insertedId: ObjectId('6961277dc5664a2aab1e2625')
}
college> db.faculty.insertOne({name: "Dr. Osborn", dept: "CSE"})
{
  acknowledged: true,
  insertedId: ObjectId('69612785c5664a2aab1e2626')
}
```

- Insert Multiple Documents

```
college> db.students.insertMany([
... { name: "Kay", age: 20, dept: "IT", marks: 88 },
... { name: "Zen", age: 22, dept: "ECE", marks: 81 }
... ])
```

```
college> db.students.insertMany([
... {name: "May", age: 21, dept: "IT", marks: 77},
... {name: "Ben", age: 19, dept: "CSE", marks: 99}
... ])
```

- MongoDB auto-generates `_id`
- Collection created automatically if not exists

4. READ Operation (Find Documents)
   - Find All Documents

```
college> db.students.find()
```

   - Find with Condition

```
college> db.students.find({ department: "CSE" })
```

   - Find Specific Fields

```
college> db.students.find(
... {dept: "ECE"},
... {name: "Zen", marks: 81}
... )
```

5. Comparison Operators

```
college> db.students.find({ marks: {$gt: 80} })
```

Operator	Meaning
`$gt`	Greater than
`$lt`	Less than
`$gte`	≥
`$lte`	≤
`$ne`	Not equal

Try all operators

6. UPDATE Operation
   - Update One Document

```
college> db.students.updateOne(
... {name: "Kay"},
... {$set: {marks: 90} }
... )
```

   - Update Multiple Documents

```
college> db.students.updateMany(
... {dept: "CSE"},
... {$set: { status: "Active"}}
... )
```

   - $set updates only specific fields
   - Without $set, entire document is replaced

7. DELETE Operation
   - Delete One Document

```
college> db.students.deleteOne({name: "Kay"})
{ acknowledged: true, deletedCount: 1 }
college> db.students.find()
[
  {
    _id: ObjectId('6961a24bc0637b28d01e2626'),
    name: 'Zen',
    age: 22,
    dept: 'ECE',
    marks: 81
  },
  {
    _id: ObjectId('6961a78ac0637b28d01e2627'),
    name: 'May',
    age: 21,
    dept: 'IT',
    marks: 77
  },
  {
    _id: ObjectId('6961a78ac0637b28d01e2628'),
    name: 'Ben',
    age: 19,
    dept: 'CSE',
    marks: 99,
    status: 'Active'
  }
]
```

- Delete Multiple Documents

```
college> db.students.deleteMany({})
{ acknowledged: true, deletedCount: 3 }
college>
```

Operation	Command
Create	`insertOne()`, `insertMany()`
Read	`find()`
Update	`updateOne()`, `updateMany()`
Delete	`deleteOne()`, `deleteMany()`

8. db.getCollectionNames()

   It **displays all collections** present in the **current database**

   Output is shown as an **array (list)**

   Used inside the **MongoDB shell**

```
college> db.getCollectionNames()
[ 'students' ]
```

9. Create Collections

```
college> db.getCollectionNames()
[ 'students', 'faculty' ]
college> D
```